

Introduction to Embedded Systems

What an embedded computer actually is, and why anybody bothers putting one inside a hot plate

Textbook: Dean, *Embedded Systems Fundamentals*, 2nd ed., Chapter 1, printed pages 3–24.

Lab manual: Lab 01 (Git and VS Code), Lab 02 (board bring-up), Manual Chapter 2 (STM32F3 Board).

How to use this set of notes

Before we start, a word about what you are holding. These notes run one file per teaching week, sixteen of them, following the lecture syllabus rather than the book's own chapter order. Each one is built the same way: a long walkthrough that assumes you know nothing about the topic and builds up, then a one-page cheat sheet at the very end that assumes you have read the walkthrough and just want the facts back fast the night before an exam.

There is one wrinkle you need to know about, and it is worth naming now rather than letting it ambush you in week eleven. Your textbook is written around the **STM32F091RC**, a Cortex-M0 part sitting on a Nucleo board. Your lab bench has an **STM32F303VCT6**, a Cortex-M4 part sitting on an F3 Discovery board. These are cousins, not twins. The ideas transfer perfectly. The register names, the peripheral counts, and quite a lot of the bit positions do not.

So every time a register shows up, you will get a purple box like this one, and it will show you both.

Book board vs your board

The book teaches the F091RC. Your bench has the F303VCT6. Whenever a specific register, bit field, or peripheral count differs between the two, this box will show you the book's version and yours side by side. Concepts first, silicon second. That is the whole rule.

Green boxes tell you where the idea shows up in your lab work. Blue boxes are definitions worth memorising. Rust-coloured boxes are traps, which is to say the specific wrong thing that a reasonable person believes right before they get it wrong.

Alright. Let's begin.

1 What we are actually after this week

Chapter 1 is the chapter everybody skims, and skimming it is a mistake, because it is the only place in the whole book that answers the question *why*. Every other chapter answers *how*. So here is the skeleton of what we are going to work through:

1. A hot plate. A real one, with a knob and a heating element, and the question of why its temperature control is bad.
2. What electronics buys you over a bent piece of metal, and what a computer buys you over plain electronics.
3. The block diagram of an actual embedded system, piece by piece, including the two pieces students always forget.

4. Two more systems, a quadcopter and a refrigerator, so we can see what changes and what stays the same.
5. The five attributes that separate an embedded system from the laptop you are reading this on.
6. The constraints, which is where the engineering actually lives.
7. The platform stack: processor core, then microcontroller, then development board. Three different things that beginners smash into one.

Items 5, 6, and 7 matter most for your midterm, because they are the ones that turn into short-answer questions. Item 1 matters most for actually understanding the rest of the course, because the hot plate is the running example the book keeps reaching back to. Items 3 and 4 will get lighter treatment, but they are not skipped, because item 3 is quietly the reason your motor driver exists in the self-balancing robot project.

2 Deep dive 1: The hot plate, and why its control is bad

Say you have an electric hot plate. Knob on the front, off through low, med, high. Underneath the shiny metal circle is a heating element, and there is a little red lamp wired so that it glows whenever the element has power flowing through it. Nothing exotic. You have used one.

Now, how does it hold a temperature? There is no thermometer in there, and there is certainly no computer. What there is instead is a **thermostatic switch**, and the mechanism is genuinely charming in its simplicity. One of the two switch contacts is made from two different metals bonded together. Different metals expand at different rates when heated, so as the contact warms up, one side stretches more than the other and the whole strip bends. Heat it enough and it bends far enough to break contact, cutting power to the element. The element cools, the strip cools, the strip bends back, contact is made, power returns. The knob does exactly one thing: it changes the physical gap between the contacts, which changes how far the strip has to bend, which changes the temperature at which the switch trips.

That is the entire control system. A piece of bent metal that acts as its own sensor, its own comparator, and its own switch, all at once.

Bang-bang control

A controller with exactly two output states, fully on or fully off, that flips between them when the measured quantity crosses a threshold. Cheap, simple, and inherently unable to settle at the target value. It can only oscillate around it.

So how well does it work? Someone stuck a thermocouple on the cooking surface and recorded it. Set the knob to Low at $t = 0$ and here is what happens.

The surface climbs to nearly 300°C over the next five minutes. Not 190°C , which is roughly where it eventually settles. Nearly 300°C . The element shuts off briefly at $t = 4:00$ and then for a longer stretch at $t = 5:00$, and the plate then spends the next ten minutes bleeding heat back out, reaching about 150°C at around $t = 14:40$. Only after that does it settle into a rhythm, cycling on and off with a period of roughly two minutes, oscillating inside a band about 20°C wide centred near 190°C .

Read that again, because there are two separate failures hiding in it. The first is that it took fifteen to twenty minutes to stabilise, during which the temperature swung by a hundred and fifty degrees. The second is that even once stabilised, it never actually holds a temperature - it just rises and falls through a 20°C range forever.

Why is it this bad? Four reasons, and they are worth separating carefully because each one gets fixed by a different part of your future design.

1. **The switch does not measure what you care about.** You care about the temperature of the cooking surface. The switch measures a blend of the metal shield's temperature and the internal air temperature. Those are correlated with the surface temperature, sure, but they are not it.
2. **There is a transport delay.** Heat has to physically conduct from the shield and the air into the switch contact before the contact can respond. By the time the strip knows things are hot, they have been hot for a while. This is the single biggest cause of that 300 °C overshoot.
3. **The output is binary.** The element is either at full power or at zero. There is no "a little bit of heat". So the system can only ever hunt back and forth across the set point.
4. **Nothing is calibrated.** The knob says low, med, high. It does not say 180 °C. There is no number anywhere in the system, which means there is no way to ask for a specific temperature and no way to check whether you got it.

Beware the trap

It is tempting to say the hot plate is bad because it has no computer. That is not quite right, and the distinction matters. Failures 1 and 2 are *sensing* problems and would be fixed by a better-placed, faster sensor even with no computer at all. Failure 3 is an *actuation* problem. Only failure 4, and the smarter control that failure 4 blocks, genuinely needs computation. Keep sensing, actuation, and computation as three separate columns in your head. Nearly every embedded design question is really a question about which column is the bottleneck.

3 Deep dive 2: What electronics buys you, and what a computer buys you on top

Let's fix the thing, in two stages, so we can see clearly where the computer starts earning its keep.

3.1 Stage one: electronics, no computer yet

Swap the bent metal strip for an electronic temperature sensor. Immediately, three things improve. The sensor is physically smaller, so it has less thermal mass and changes temperature faster, which shrinks the delay. You can mount it directly on the bottom of the hot plate surface instead of off in the air somewhere, which means it now measures the thing you actually care about. And you can switch the heating element on and off far more often than a mechanical strip can, which shrinks those 20 °C ripples down to something much tighter.

Push that switching frequency high enough and something qualitatively new appears. If you are flipping the element on and off many times per second, the plate's own thermal inertia smooths it out, and what the food feels is the *average* power. Now you are not choosing between full-on and full-off any more. You are choosing what fraction of the time the element is on, and that fraction can be anything from 0 to 1.

If that smells like PWM to you, good, that is exactly what it is, and you will meet it properly in week 13. Hold that thought.

3.2 Stage two: add the computer

Now the sensor gives you an actual number rather than an above-or-below verdict, and the output accepts an actual number rather than an on-or-off command. With a number coming in and a number going out, you can compute.

And once you can compute, the obvious move is: measure how far the actual temperature is from the desired temperature, and turn the heater on *in proportion* to that gap. Far below the set point, heat hard. Slightly below, heat gently. The overshoot largely disappears, because the controller eases off before it arrives instead of slamming into the target at full power.

Error, and the loop around it

$\text{error} = \text{set point} - \text{measured value}$. The set point is what you asked for, the measured value is what you got, and essentially every control system in this course exists to compute that difference and shrink it. Your self-balancing robot's PID controller is this same idea with two extra terms bolted on.

Here is the part that students underrate. Once an MCU is in the box, the marginal cost of a new feature drops to almost nothing, because a feature is now just more software. The book lists a few for the hot plate and they are worth reading as a pattern rather than a list:

- Auto-shutoff after a fixed period with no knob activity. That is a *safety* feature, and it costs one timer.
- A knob marked in real temperatures instead of low/med/high. That is a *usability* feature, and it costs a calibration table.
- Flashing the indicator lamp when the plate is near the set point. That is a *feedback* feature, and it costs about four lines of code.
- A panel showing current and target temperature together. That is a *diagnostics* feature, and it costs a display driver.

Notice the economics. The hardware was already paid for. Each of those features would have needed its own dedicated mechanism in the pre-computer design, and now they share one. This is the actual argument for embedded computing, and it is a cost argument far more than a capability argument: MCUs are cheap because they are made by the hundred million, they arrive with the interface circuitry already inside them, and so the dominant cost of an embedded product ends up being the software development, not the silicon.

4 Deep dive 3: How you actually embed the computer

Right, so we have decided to put an MCU in the hot plate. What does the block diagram look like? Let's walk the signal from the world, through the chip, and back out to the world.

Inputs. Two of them. The knob position, which encodes the desired temperature, and the sensor output, which encodes the actual temperature. The sensor gives you a voltage that varies proportionally with temperature. But the MCU is a digital device and cannot think about voltages, only about numbers. Something has to convert one into the other, and that something is the ADC. Week 11.

Computation. The MCU subtracts, decides, and produces an output command. This is the only part that is pure software, and it is also the only part you can change after the product ships.

Outputs. A signal to switch the heating element. And here is the first piece people forget: an MCU output pin puts out about 3.3 V and can source a few milliamps. A heating element wants 110 V or 220 V at several amps. Those numbers are not in the same universe. So you need a **driver circuit** sitting between the pin and the element, something that takes the MCU's polite little logic-level nudge and uses it to gate a great deal of power.

In the lab

This exact gap is why the Balance Car Shield on your self-balancing robot carries a **TB6612FNG** motor driver. Your STM32F303 cannot drive a Chihai CHR-GM37-520 gearmotor directly. Not "should not", *cannot* - the pin would fail. The MCU produces PWM at logic level, the TB6612FNG uses that to switch the 12 V motor supply. Same pattern as the hot plate's driver circuit, same reason.

Power supply. The second forgotten piece. Line voltage comes in at 110 or 220 V. The MCU needs 3.3 V. A power supply has to bridge that, and it does not usually appear on the block diagram even though the system is a paperweight without it.

Three definitions to fix now

Integrated circuit (IC): an electronic circuit with its components built into a single piece of silicon.

Central processing unit (CPU): the hardware that fetches, decodes, and executes instructions.

Microcontroller unit (MCU): an IC containing a CPU, *plus* peripherals, *plus* support circuits, and usually memory as well. The CPU is a component of the MCU, not a synonym for it.

5 Deep dive 4: Two more systems, for contrast

One example is an anecdote. Three is a pattern. So, briefly:

The quadcopter. A user holds a remote and asks for things like "go forward" or "flip over". But the aircraft does not have a forward motor. It has four rotors, and forward motion is what happens when you make the rear pair spin slightly faster than the front pair, and the correct amount of "slightly" depends entirely on how the aircraft is currently tilted and moving, which changes in milliseconds. So the embedded computer's job is translation: user intent in, four individual motor speeds out. To do that translation it needs to know its own orientation, which is why there are accelerometers measuring linear acceleration along three axes and gyroscopes measuring rotation about three axes: roll, pitch, and yaw.

Look at the board inside one and you will find the MCU, the sensors, LEDs, wires out to four motors, a radio interface chip, and a scattering of little three-legged transistors, which are the motor drivers. Same driver pattern again.

In the lab

Stop and notice what you just read. Accelerometer plus gyroscope, feeding an MCU, which computes orientation and commands motor speeds, fast enough to keep an unstable object upright. That is the quadcopter and it is also, exactly, your self-balancing robot. Your LSM303AGR is the accelerometer over I²C, your I3G4250D is the gyroscope over SPI, and your two gearmotors are the rotors. The book hands you your own final project in Chapter 1 as a throwaway example.

The refrigerator. Two compartments each held in their own temperature band, user-adjustable set points, lights that come on with the door, an ice maker, a chilled water dispenser, a chime if the door hangs open too long, and a reminder when the water filter is due. Contrast it against the quadcopter: the fridge has to do many unrelated things at once but almost nothing needs to happen fast. The quadcopter does one thing but the deadline is brutal. Different problems, same chip family.

And that difference is exactly what the next section is about. Pat yourself on the back, the abstract half of the chapter is behind us.

6 Deep dive 5: The five attributes of an embedded system

What makes an embedded system a different animal from a general-purpose computer? Five things, and they show up as exam questions with reliable regularity.

6.1 Interfacing with inputs and outputs

A laptop talks to keyboards and screens. An embedded system talks to the physical world, which is analog, noisy, slow, and occasionally on fire. Inputs arrive as varying voltages from sensors, as contact closures from switches, as pulse trains from encoders. Outputs go to motors, heaters, lamps, and displays, almost always through a driver, because the MCU pin cannot supply the power. This is the entire content of weeks 2, 3, and 11.

6.2 Concurrency

The refrigerator has to hold the freezer temperature *and* hold the fridge temperature *and* watch the door *and* run the ice maker, all at the same time, with one CPU that can only execute one instruction at a time.

Concurrency

Multiple activities making progress over the same period of time. Note carefully that this is not the same as *parallelism*, which means multiple activities executing at literally the same instant. A single-core MCU gives you concurrency by interleaving. It cannot give you parallelism at all.

How you interleave those activities without an operating system underneath you is the entire subject of weeks 4 and 5, and it is where a lot of people's mental model of embedded programming finally clicks.

6.3 Responsiveness

The system has to react to an external event within some deadline that the external world sets, not one you get to negotiate. If the quadcopter takes 200 ms to notice it is tipping, it does not tip more slowly out of politeness. It crashes.

The mechanism that makes fast response possible is the **interrupt**: hardware that yanks the CPU away from whatever it was doing, runs a short handler, and puts it back. Weeks 6, 7, and 8, and honestly the most important machinery in the course.

6.4 Reliability and fault handling

Nobody reboots a refrigerator. Nobody reinstalls a pacemaker. Embedded systems are expected to run for years without intervention, which means the software must handle its own faults rather than surfacing them to a user who is not there. A hardware watchdog timer that resets the chip if the software stops petting it is the standard last line of defence, and it arrives in week 13.

6.5 Diagnostics

When the thing does break, somebody has to work out why, often in the field with no debugger attached. So the system needs to be able to report on itself: log a fault code, flash a pattern, print over a serial link. This is why your very first real lab gets UART working. Not because UART is conceptually first, but because without it you are debugging blind.

Beware the trap

Students often treat responsiveness and speed as the same thing. They are not. Responsiveness is about *worst-case latency*, the longest you might ever wait, while raw speed is about average throughput. A system that averages a 1 ms response but occasionally takes 500 ms is fast and unresponsive, and for a balancing robot that is a failure. This distinction is exactly why you will care about interrupt priorities and critical section length in week 8.

7 Deep dive 6: Constraints, where the engineering actually happens

Here is the thing that separates this course from your other programming courses. On a laptop, if a data structure is a bit wasteful, you shrug. On an MCU you cannot shrug, because the constraints are hard and they push against each other.

- **Cost.** Consumer products live and die on the bill of materials. A part that costs \$0.30 more, times ten million units, is a real number.
- **Memory.** Your F303VCT6 has 40 KB of SRAM. Total. For everything: globals, stack, heap, buffers. Allocate a careless `float` array and you can consume a measurable fraction of it.
- **Power.** Battery-operated systems are judged on how long they last, and a CPU running flat out in a busy-wait loop burns power doing nothing useful.
- **Time.** Deadlines set by physics, as discussed.
- **Size and weight.** Very much a real constraint on anything that flies or is worn.
- **Reliability and safety.** A defect in a car's brake controller is not a bug report, it is a recall, and possibly worse.

These trade against each other, always. Want faster response? Raise the clock, burn more power. Want less power? Sleep more, respond slower. Want less memory used? Compute values on demand instead of caching them, spend more time. There is no configuration that wins on every axis, and choosing which axis to lose on is what the job is.

8 Deep dive 7: The platform stack

Last section, and it is the one most worth getting precisely right, because three distinct things get muddled together constantly. Let's separate them, from the inside out.

8.1 Layer one: the processor core

At the centre is an Arm **Cortex-M** core. Arm does not manufacture chips. Arm designs the CPU core and licenses that design to companies like STMicroelectronics, who wrap their own memory and peripherals around it and fabricate the result. This is why an STM32 and an NXP Kinetis can run the same compiled code at the core level while having completely different peripherals.

The Cortex-M family is graded roughly by capability:

Core	Architecture	Notable	Typical role
Cortex-M0 / M0+	ARMv6-M	Smallest, no DSP, no FPU	Low cost, low power
Cortex-M3	ARMv7-M	Full Thumb-2, no FPU	General purpose
Cortex-M4	ARMv7E-M	DSP instructions, optional FPU	Signal processing, control
Cortex-M7	ARMv7E-M	Superscalar, caches	High performance

8.2 Layer two: the microcontroller

Wrap the core in flash memory, SRAM, and a pile of peripherals, put it all on one die, and you have an MCU. This is the part number you actually buy: STM32F091RC, or STM32F303VCT6.

Book board vs your board

	Book: STM32F091RC	Yours: STM32F303VCT6
Core	Cortex-M0	Cortex-M4
Architecture	ARMv6-M	ARMv7E-M
Max clock	48 MHz	72 MHz
FPU	None	Yes, single precision
DSP instructions	No	Yes
Flash	256 KB	256 KB
SRAM	32 KB	40 KB
NVIC priority bits	2 (4 levels)	4 (16 levels)
Bit-banding	No	Yes

Two consequences worth flagging early. First, the extra NVIC priority bits mean your interrupt priority schemes in week 7 have more room than the book's examples assume. Second, the FPU and DSP instructions are the reason Lab 07 can ask you to run a CMSIS-DSP filter at all - that lab would be painful on the book's M0.

The lab manual's own summary of your part, which is worth committing to memory since it turns up in every lab report you write:

Feature	STM32F303VCT6
Core	Arm Cortex-M4 with FPU and DSP support
Clock frequency	Up to 72 MHz
Flash memory	256 KB, non-volatile program storage
SRAM	40 KB, volatile data memory
GPIO	Up to 87 pins
Analog	4 ADCs, 2 DACs
Timers	Advanced, general purpose, and basic
Communication	UART, SPI, I ² C, USB
Debugger	ST-LINK/V2 integrated, via mini-USB
Operating voltage	2.0 V to 3.6 V

8.3 Layer three: the development board

An MCU on its own is a chip with legs. To do anything with it you need it soldered to a board with a power supply, a crystal, a programming and debug interface, and connectors bringing the pins out where you can reach them. That is a development board.

The book uses a **Nucleo-F091RC**. You have an **STM32F3 Discovery**. Discovery boards are ST's "here is the chip plus some toys" line, which is why yours arrives with sensors already soldered on.

Your board carries:

- Four user LEDs and a user push-button
- A MEMS accelerometer, **LSM303AGR**, on I²C1
- A MEMS gyroscope, **I3G4250D**, on SPI1
- A USB interface for power, programming, and communication
- An integrated ST-LINK/V2 debugger

Beware the trap

Your lab manual is internally inconsistent about the on-board sensors. Some sections name the LIS302DL accelerometer and LSM303DLHC gyroscope, which were fitted to older revisions of this board. The current revision, and the one your own project plan correctly identifies, carries the **LSM303AGR** and the **I3G4250D**. Before Lab 08 and Lab 09, read the silkscreen on your actual board rather than trusting either the manual or these notes. Note also that the manual sometimes calls the LSM303DLHC a gyroscope; it is an e-compass, an accelerometer plus magnetometer. Datasheets over documents, always.

HAL and BSP, since you will use both

HAL (Hardware Abstraction Layer) gives you generic, chip-level peripheral functions. It knows what a GPIO port is. It does not know what is wired to it.

BSP (Board Support Package) sits on top of HAL and gives you board-specific functions. It knows that on *this* board, LED3 happens to be on a particular pin, so you can write `BSP_LED_Toggle(LED3)` without looking it up.

Listing 1: The smallest program that proves your board is alive. Lab manual, Section 2.3.3.

```

1 | #include "stm32f3_discovery.h"
2 |
3 | int main(void) {
4 |     HAL_Init();           // bring up the HAL library
5 |     BSP_LED_Init(LED3);   // board knows which pin LED3 is on
6 |     while (1) {
7 |         BSP_LED_Toggle(LED3);
8 |         HAL_Delay(500);   // busy-wait, 500 ms
9 |     }
10 | }
```

Look at that `HAL_Delay(500)` and file a complaint against it in your head now. It works, and it is also the CPU sitting in a loop doing absolutely nothing for half a second, which is exactly the sin that weeks 4, 5, and 12 exist to correct.

9 Where this lands in your lab work

In the lab

Lab 01, Git and VS Code, 4%. No microcontroller at all. You install Ubuntu or set up a VM, install VS Code and Git, authenticate the GitHub CLI with `gh auth login`, and create your first repository. This feels like administrative overhead and it is not: part of every single lab grade for the rest of the semester is allocated to proper Git usage, meaning real commit messages and regular pushes rather than one giant "final" commit at midnight. You also install STM32CubeMX and the ST-LINK drivers here.

Lab 02, board bring-up and UART, 6%. First contact with the hardware. Plug in over the ST-LINK USB connector and confirm LD1 and LD2 light up, which tells you the board has power and the debugger is alive. Then you get UART running and print from the chip to your PC. Everything in this week's Section 6.5 about diagnostics is why this lab is second rather than tenth.

Power warning, from the manual and worth repeating. Power the board through the ST-LINK USB connector. Do not feed voltage into the 3V or 5V pins unless an experiment explicitly requires it and an instructor is watching. When those pins are used as outputs, anything you hang off them must stay under 100 mA.

10 Tying it back

We opened with a question: why does anybody put a computer inside a hot plate? Here is the answer assembled from everything above.

Because a bent piece of metal can only sense one thing, in the wrong place, slowly, and can only respond in a binary way, so it overshoots by a hundred degrees and then oscillates forever. Electronics fixes the sensing and makes the actuation proportional. A computer fixes what you do with the sensing, by letting you compute an error and act in proportion to it, and then, almost for free, it lets you add safety timeouts and calibrated set points and status displays, because once the chip is in the box a new feature is just more software.

And every single thing that made the hot plate work is a topic in this course. The sensor becomes the ADC in week 11. The proportional output becomes PWM in week 13. Reacting to the knob without missing a temperature sample becomes concurrency in weeks 4 and 5, and interrupts in weeks 6 through 8. Reporting what the controller is thinking becomes UART in week 14. Not falling over when something goes wrong becomes the watchdog in week 13.

The whole syllabus is one hot plate, taken apart.

Cheat sheet: Week 01

Core definitions

Embedded system = a computer placed inside a larger system to improve it, usually by controlling it.

IC = circuit built into one piece of silicon. **CPU** = fetch, decode, execute hardware. **MCU** = IC containing CPU + peripherals + support circuits + usually memory.

Error = set point – measured value. Every controller in this course shrinks this.

The hot plate, in numbers

Thermostatic switch, bimetallic contact, bang-bang control. Overshoots to $\approx 300^\circ\text{C}$ in 5 min, settles only after 15–20 min, then oscillates in a $\approx 20^\circ\text{C}$ band around 190°C with a ≈ 2 min period.

Four faults: measures the wrong thing · transport delay to the sensor · binary output · no calibration.

Five attributes of an embedded system

1. **I/O interfacing** - talks to the physical world, needs drivers for power.
2. **Concurrency** - many activities progressing together on one CPU. Not parallelism.
3. **Responsiveness** - meets externally imposed deadlines. Worst-case latency, not average speed.
4. **Reliability** - runs unattended for years, handles its own faults. Watchdog.
5. **Diagnostics** - can report on itself when it breaks. UART.

Constraints that trade against each other

Cost · memory · power · time · size and weight · reliability and safety.

No design wins on all six. Naming which one you sacrificed is the answer examiners want.

Platform stack, inside out

Core (Arm Cortex-M4, licensed design) → **MCU** (STM32F303VCT6, core + flash + SRAM + peripherals) → **Board** (STM32F3 Discovery, MCU + power + debugger + sensors + connectors).

Your part: Cortex-M4, FPU + DSP, 72 MHz max, 256 KB flash, 40 KB SRAM, up to 87 GPIO, 4 ADC, 2 DAC, ST-LINK/V2 on board, 2.0–3.6 V.

Book's part: STM32F091RC, Cortex-M0, 48 MHz, 256 KB flash, 32 KB SRAM, no FPU, no DSP.

On-board sensors: LSM303AGR accelerometer on I²C1, I3G4250D gyroscope on SPI1. Verify against the silkscreen.

HAL = generic peripheral functions. **BSP** = board-specific, calls HAL underneath.

Likely exam prompts from this chapter

- Distinguish an embedded system from a general-purpose computer. Answer with the five attributes.
- Explain why bang-bang control cannot hold a set point. Answer with overshoot and transport delay.

- Distinguish concurrency from parallelism on a single-core MCU.
- Distinguish CPU, MCU, and development board.
- Given a system, list its inputs, outputs, and the interface hardware each one needs.